Introduction to Diffusion Language Models

Woojin Kim

SNU AIDAS Lab
wjk9904@snu.ac.kr





AIDAS Lab

Table of contents

1 The Diffusion Breakthrough

2 Revolutionizing Vision with Diffusion

3 Inside the Diffusion Process

4 Bringing Diffusion to Language

5 Diffusion Language Models Today

6 Seminar Schedule

Goals of the Seminar

- To understand the foundational formulas behind diffusion models
- To explore the key components that make up a diffusion model
- To gain insight into how diffusion models can be integrated into one's own research

The Diffusion Breakthrough

Rethinking Language Generation

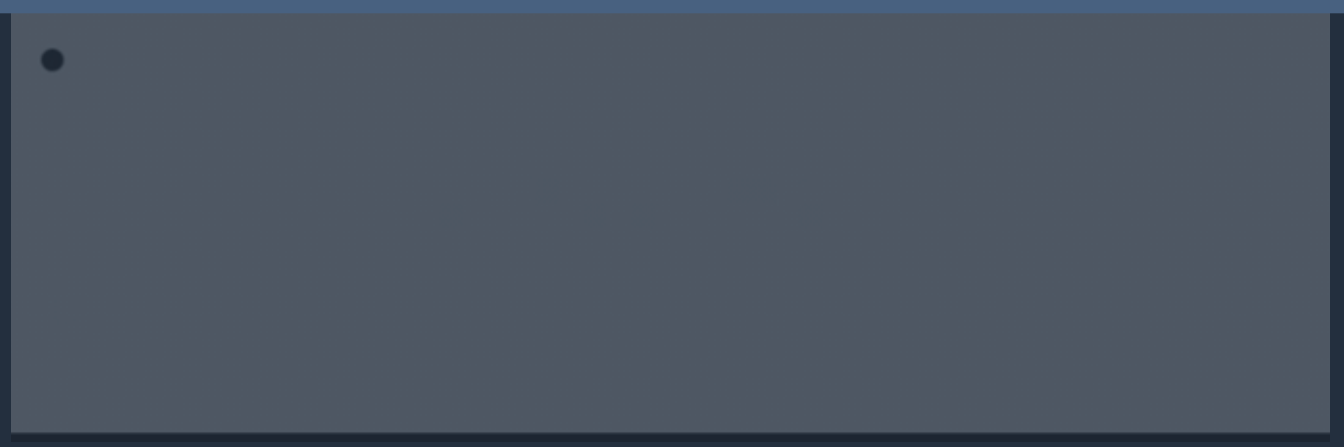
- Auto-regressive models (GPT, Qwen) currently dominate the field, but they generate text one token at a time. Is this truly the most optimal approach?



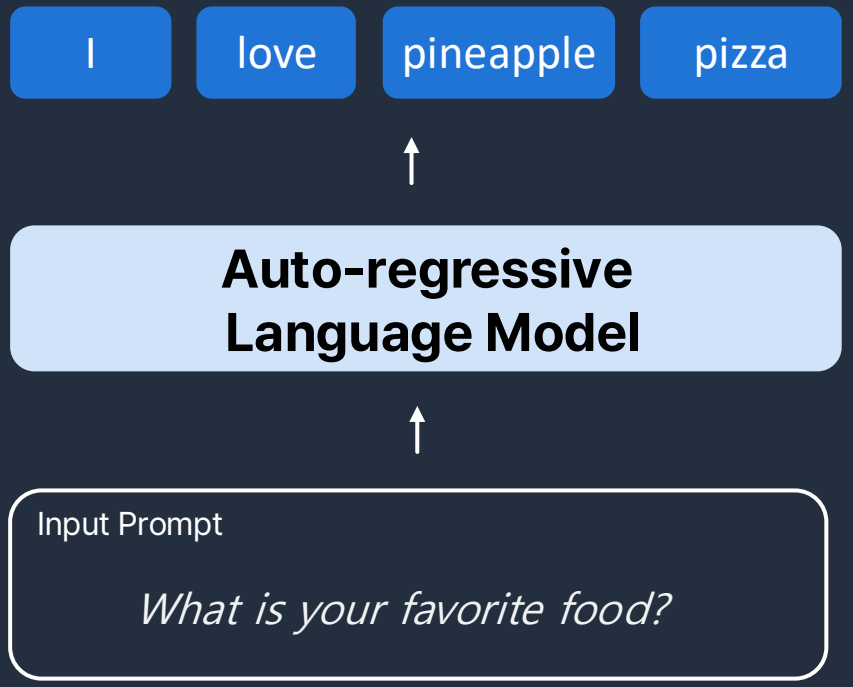
Rethinking Language Generation

- Auto-regressive models (GPT, Qwen) currently dominate the field, but they generate text one token at a time. Is this truly the most optimal approach?

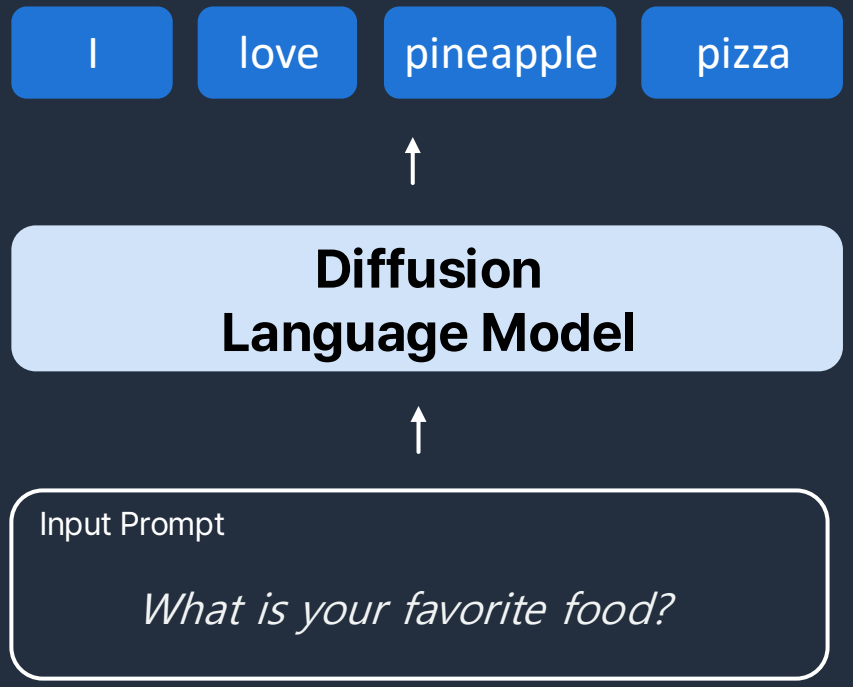
"What if we could generate coherent paragraphs without going one token at a time?"



A New Paradigm: Diffusion-based generation



One token at a time

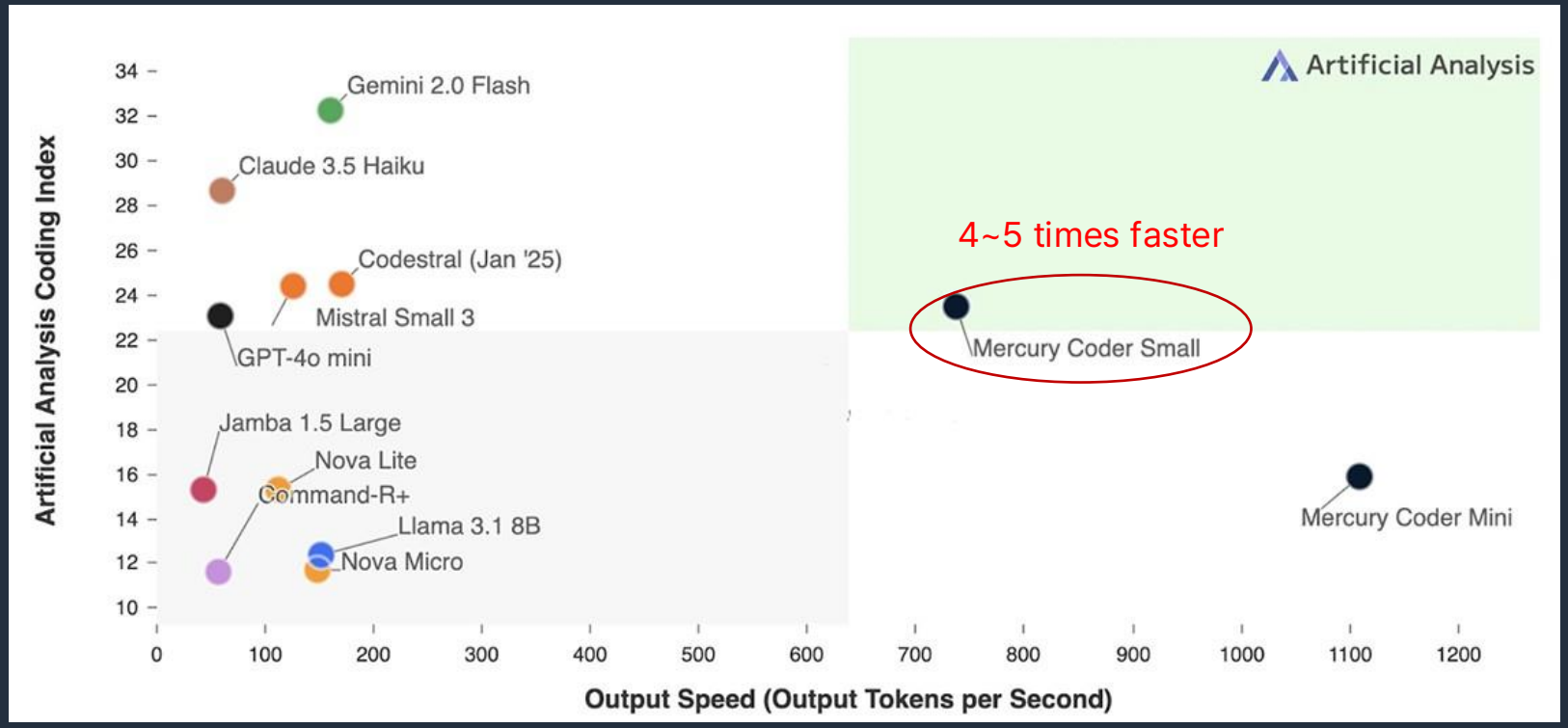


Multiple tokens at a time

Mercury Coder

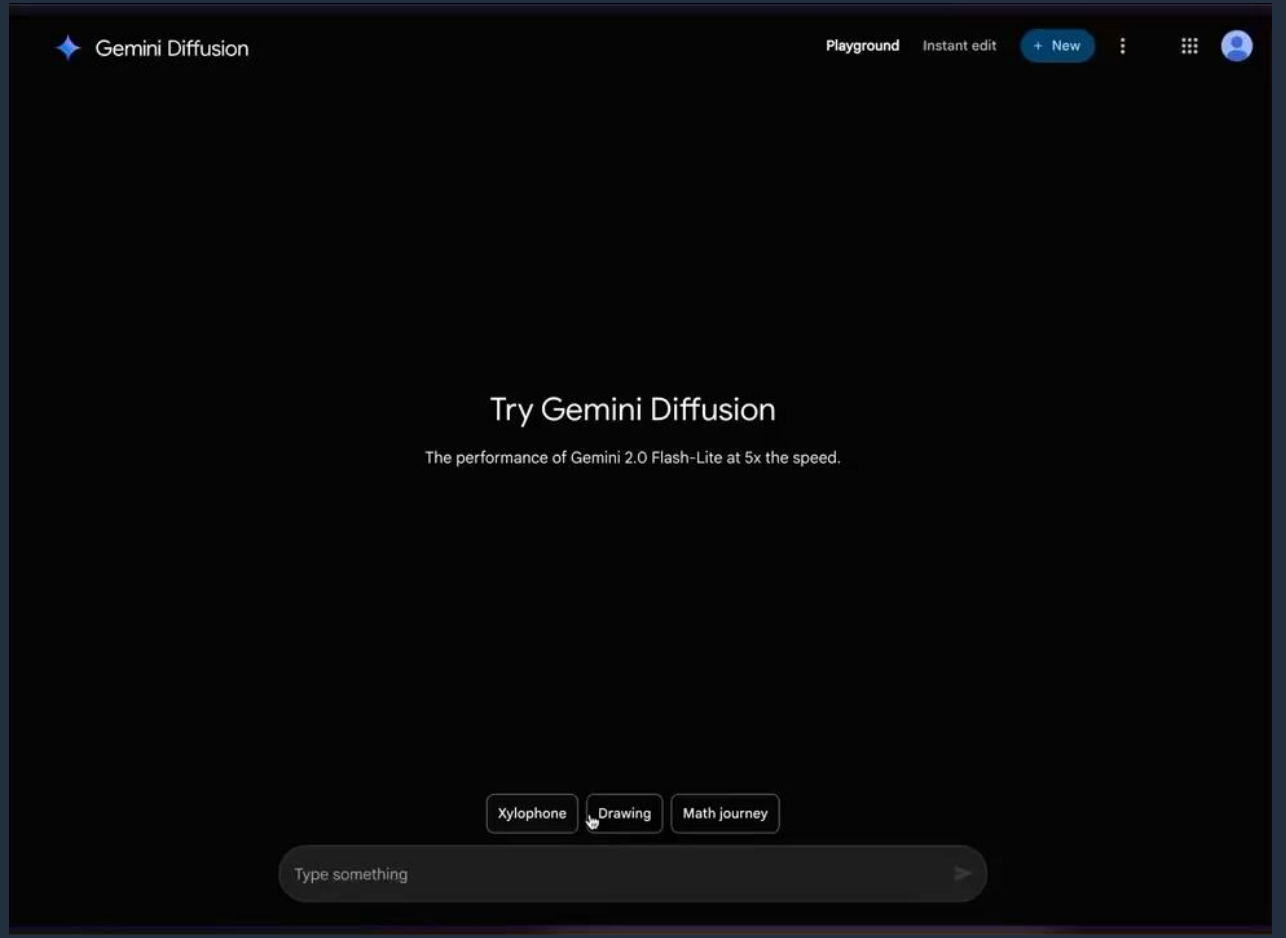


Mercury Coder



(https://www.inceptionlabs.ai/introducing-mercury)

Gemini Diffusion



Gemini Diffusion

performance is comparable to much larger models, whilst also being faster.

Benchmark	GEMINI DIFFUSION	GEMINI 2.0 FLASH-LITE
Code LiveCodeBench (v6)	30.9%	28.5%
Code BigCodeBench	45.4%	45.8%
Code LBPP (v2)	56.8%	56.0%
Code SWE-Bench Verified*	22.9%	28.5%
Code HumanEval	89.6%	90.2%
Code MBPP	76.0%	75.8%
Science GPQA Diamond	40.4%	56.5%

Gemini Diffusion speed

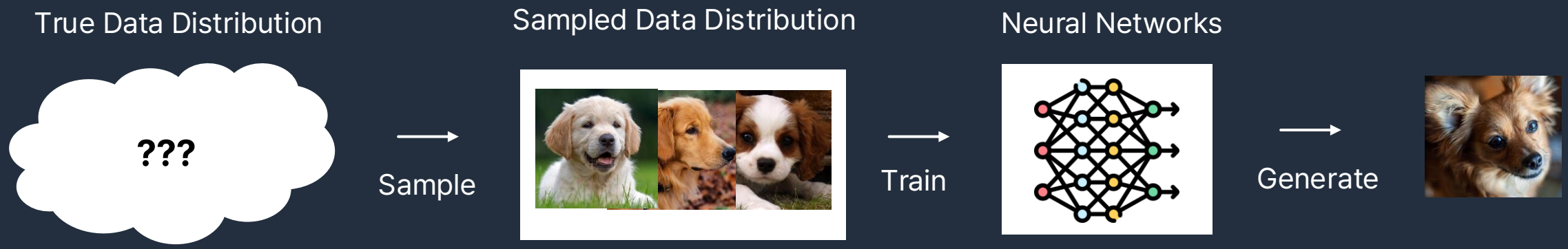
Sampling speed excluding overhead	1479 tokens / sec
Overhead	0.84 sec

Revolutionizing Vision with Diffusion

Deep Generative Models

Generative Models: Models that learn data patterns to generate new, realistic examples.

Objective: To train a model $p(x)$ that approximates the true data distribution $P_{data}(x)$



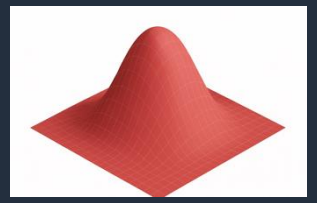
Deep Generative Models

Even though we can sample data, we want to **train a model** that learns to generate it

However, this is hard because:

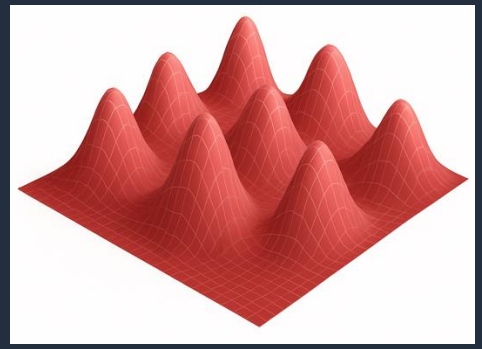
- Real data lies in a **high-dimensional, complex** space
- We usually start from a simple distribution (like a standard Gaussian)
- Learning the transformation from a simple latent space to real data is non-trivial

Simple prior



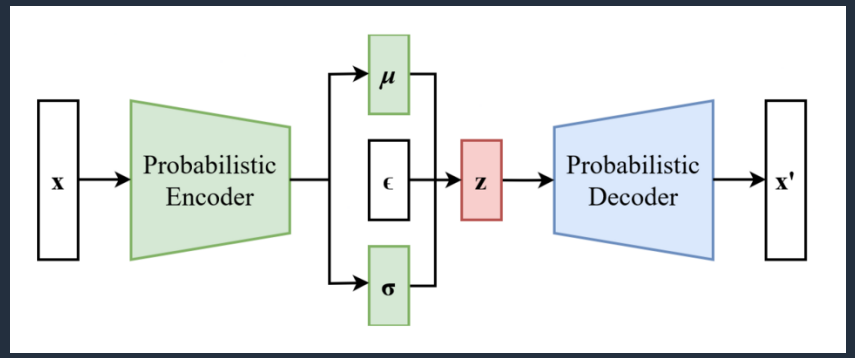
learning direct transformation is hard

Complex data distribution



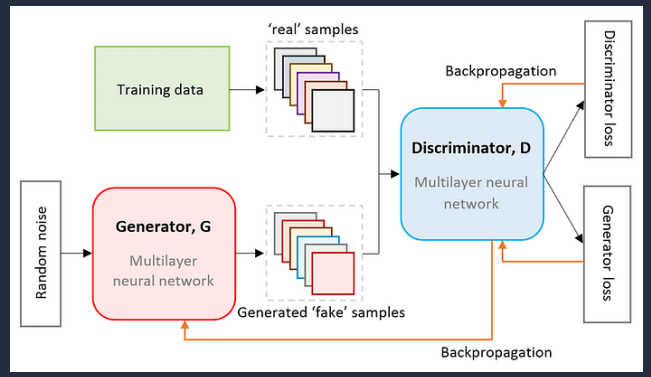
Deep Generative Models

VAE (Kingma & Welling, 2013)



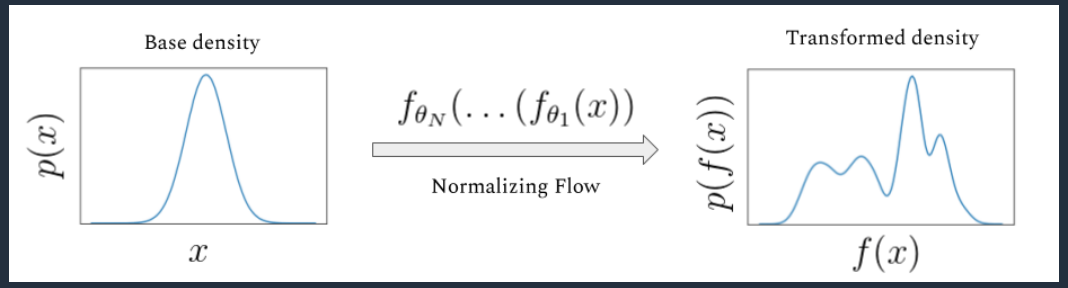
(https://commons.wikimedia.org/wiki/File:Reparameterized_Variational_Autoencoder.png)

GAN (Goodfellow et al., 2014)



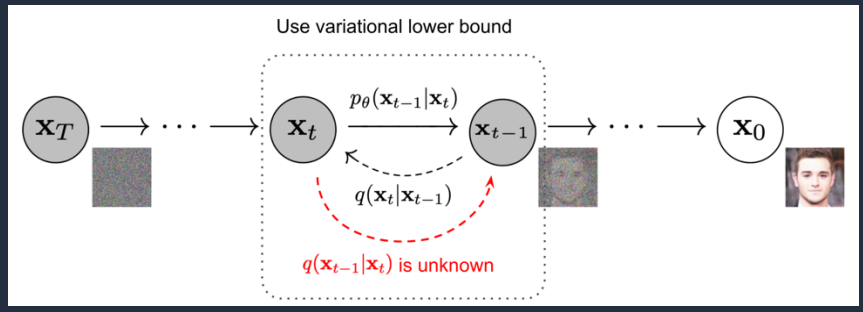
(https://arxiv.org/abs/2112.01925)

Normalizing Flow (Rezende & Mohamed, 2015)

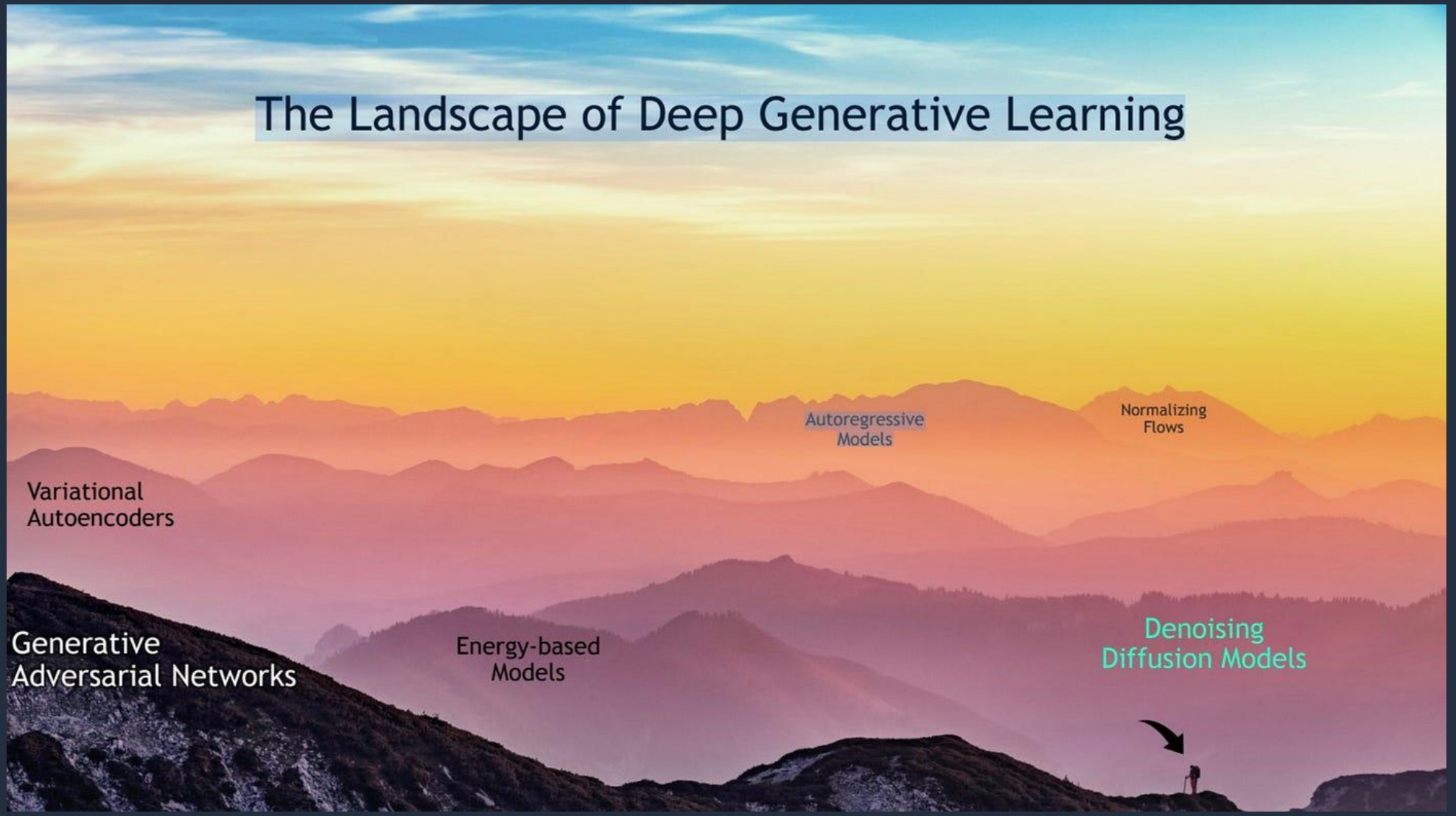


(https://siboehm.com/articles/19/normalizing-flow-network)

Diffusion Model (Ho et al., 2020)

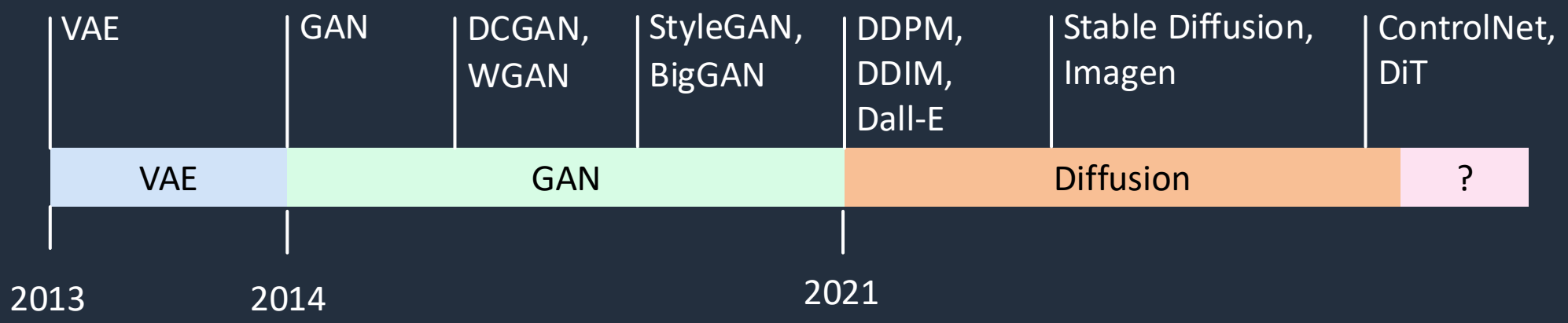


(https://arxiv.org/abs/2006.11239)



(CVPR 2022 Tutorial: Denoising Diffusion-based Generative Modeling: Foundations and Applications)

The Era of Diffusion Model



Application: Text to Image Generation

Imagen (Google)



A small cactus wearing a straw hat and neon sunglasses in the Sahara desert.

Stable Diffusion (Stability AI)



Scene of a giant ancient tortoise with a fantasy city built on its back. The tortoise's shell ...

DaLL•E (OpenAI)



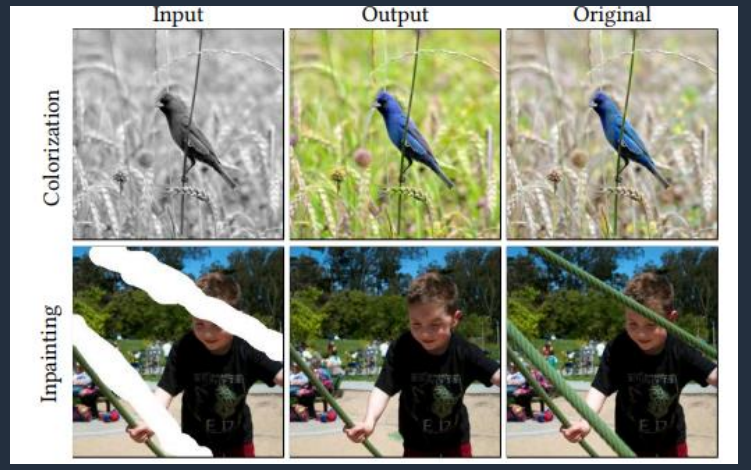
A photorealistic image of an astronaut riding a horse

Application: Impainting, Reconstruction

Repaint
(2022)



Palette
(2022)



Application: Video generation

Imagen Video
(2022)



Figure 1: Imagen Video sample for the prompt: "*A bunch of autumn leaves falling on a calm lake to form the text 'Imagen Video'. Smooth.*" The generated video is at 1280×768 resolution, 5.3 second duration and 24 frames per second.

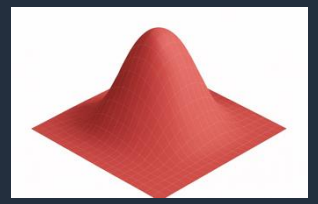
Gen Series
(Runway)



Diffusion Process

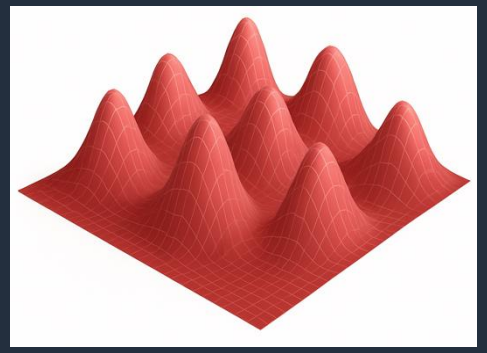
How does diffusion models work?

Simple prior



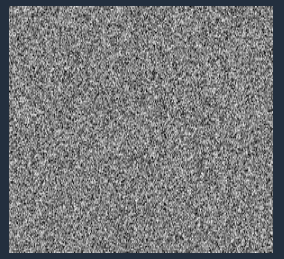
learning direct transformation is hard

Complex data distribution



How does diffusion models work?

Simple prior



But going the other way—
adding noise to data—is simple

learning direct
transformation
is hard

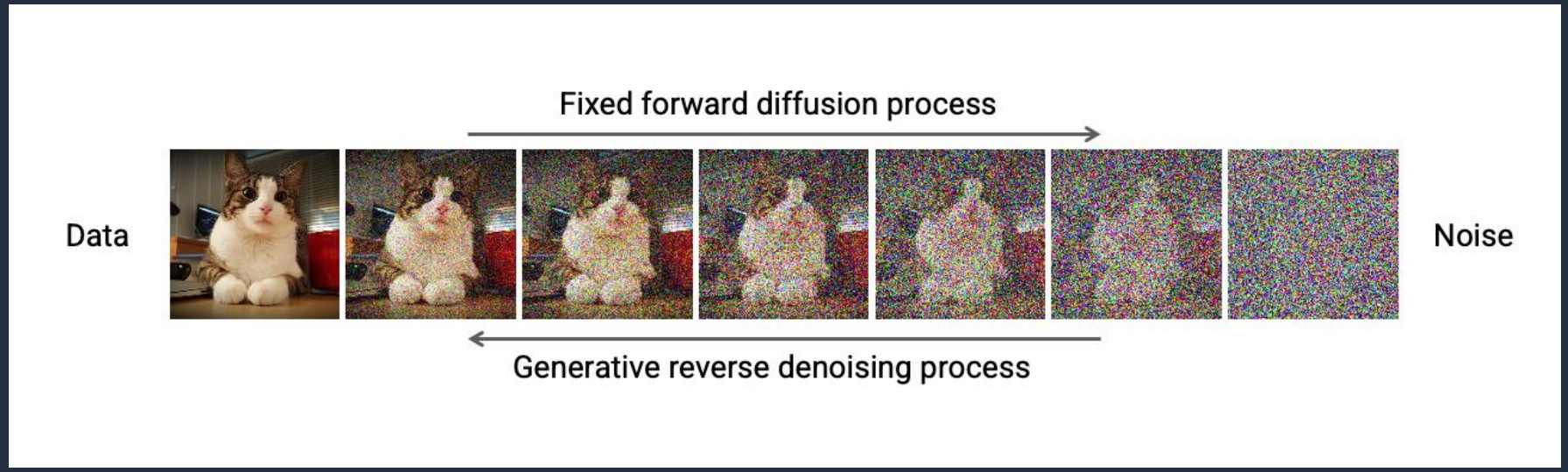
Complex data



Then, let's simply **add noise** step by step
and let the model learn how to **denoise** it, one step at a time.

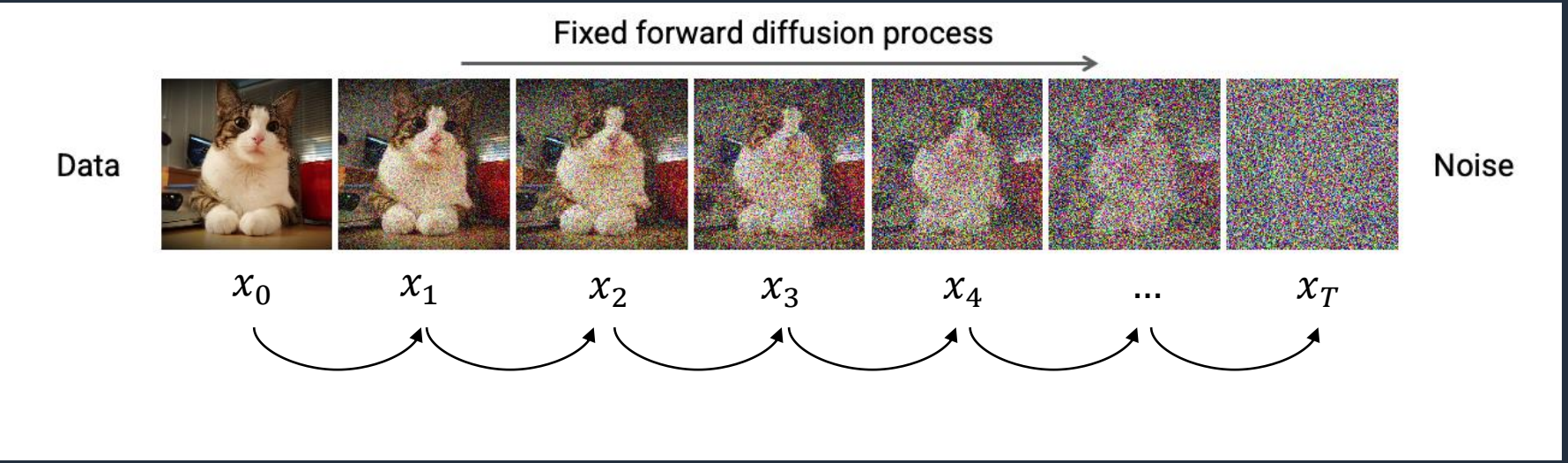
Diffusion Process

- **Forward diffusion process**: Gradually adds noise to corrupt original data
- **Backward diffusion process**: Model learns to generate data by iterative denoising



Diffusion Forward Pass

- Define the forward process with Markov chain: $q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$
- Each transition: $\mathrm{q}(\mathrm{x_t}|\mathrm{x_{t-1}}) = \mathrm{N}(\mathrm{x_t};\ \sqrt{1-\beta_t}\mathrm{x_{t-1}}, \beta_t \mathrm{I})$



- With sufficient amount of noise and timesteps, the data eventually converges to pure noise: $\mathrm{q}(x_T) = \mathrm{N}(x_T, 0, \mathrm{I})$

Diffusion Backward Pass

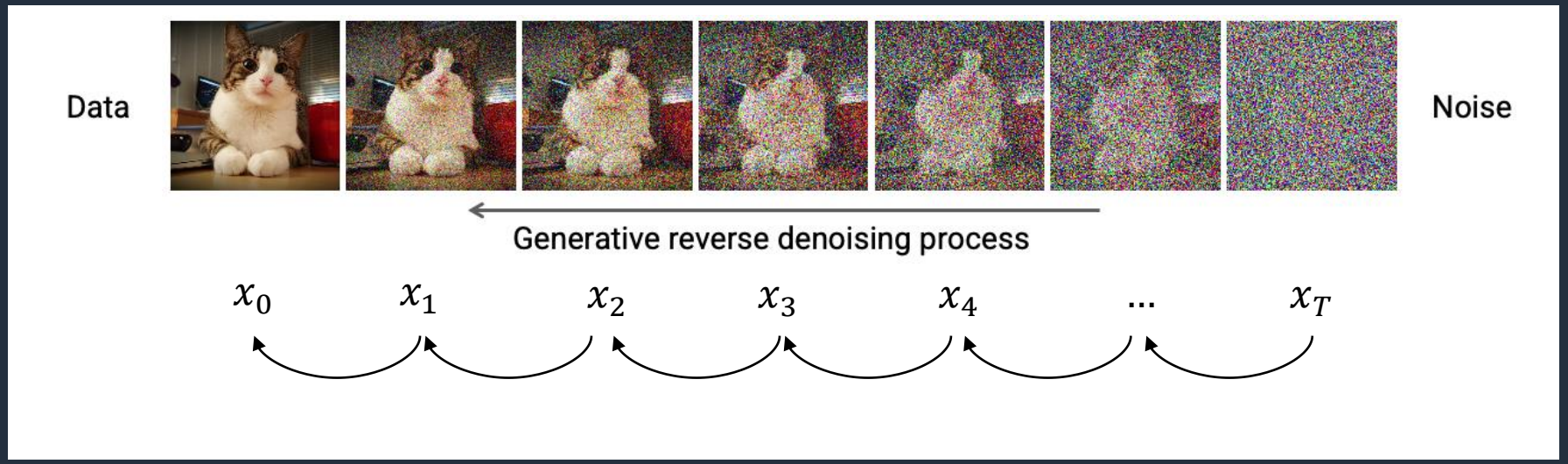
- So, if everything becomes noise… how do we get the data back?
- Begin with $\mathrm{q}(x_T) = \mathrm{N}(x_T, 0, \mathrm{I})$
- Iteratively denoise with True denoising distribution: $\underbrace{\mathrm{q}(\mathrm{x}_{t-1}|\mathrm{x}_t) \propto q(x_{t-1})q(x_t|x_{t-1})}_{\text{Intractable}}$

-> Let's approximate the reverse process $\mathrm{q}(\mathrm{x}_{t-1}|\mathrm{x}_t)$ with Gaussian.

Diffusion Backward Pass

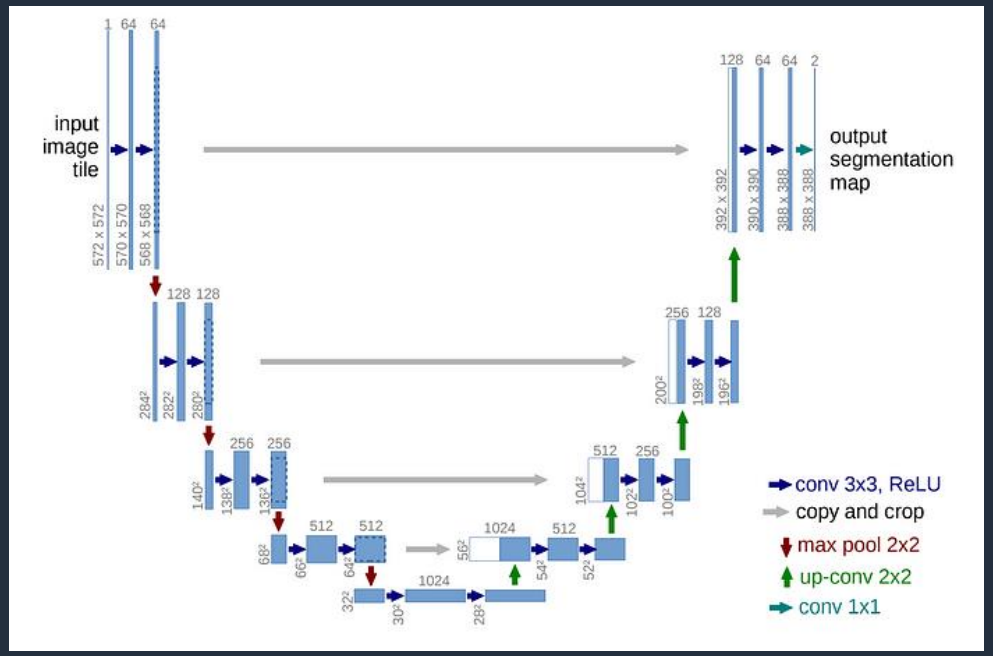
- Begin with $\mathrm{p}(x_T) = \mathrm{N}(x_T, 0, \mathrm{I})$
- Iteratively denoise with learned reverse process: $p_\theta(\mathrm{x_{t-1}}|\mathrm{x_t}) = \mathrm{N}(\mathrm{x_{t-1}};\ \underline{\mu_\theta(x_t, t)}, \sigma_\mathrm{t}{}^2\mathrm{I})$

Estimated by network

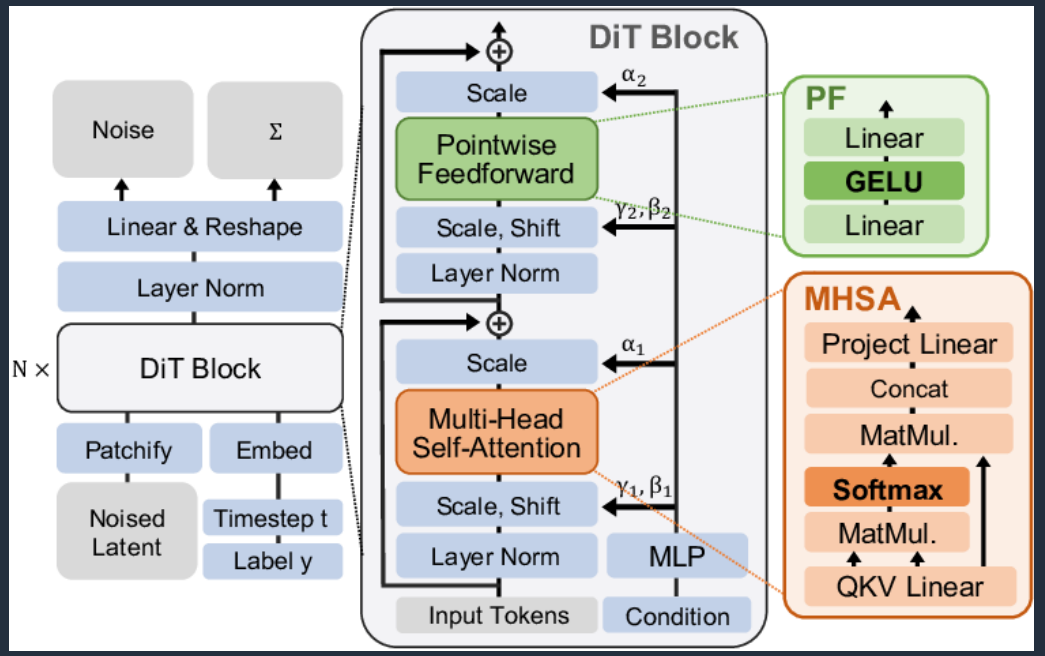


Model architecture

U-net
(Ronneberger et al., 2015)

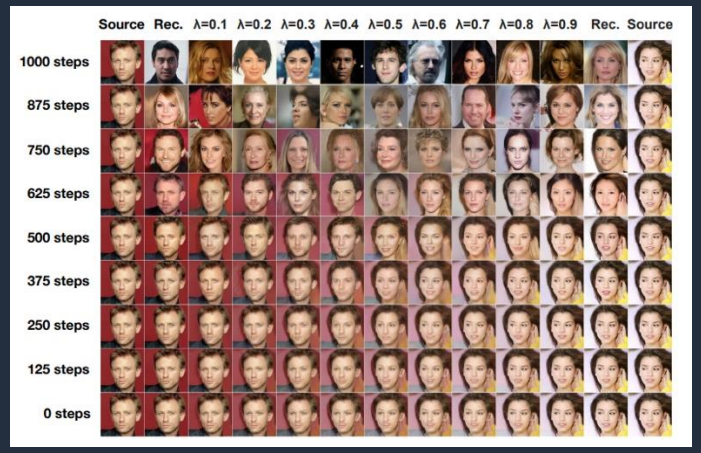


DiT
(Peebles & Xie, 2022)

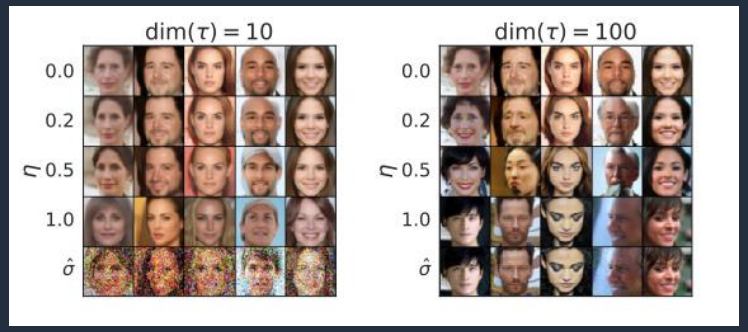


Diffusion Sampling

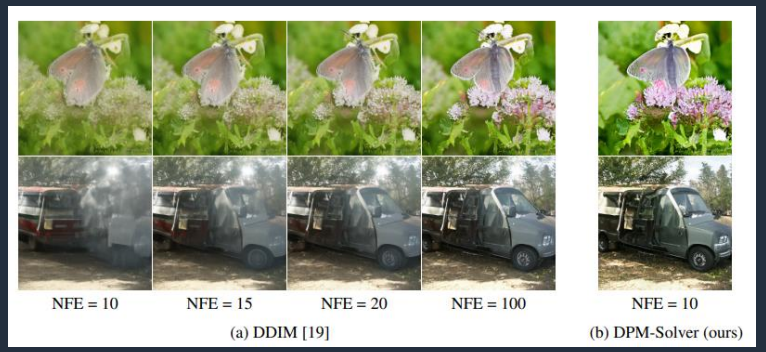
DDPM
(Ho et al., 2020)



DDIM
(Song et al., 2020)

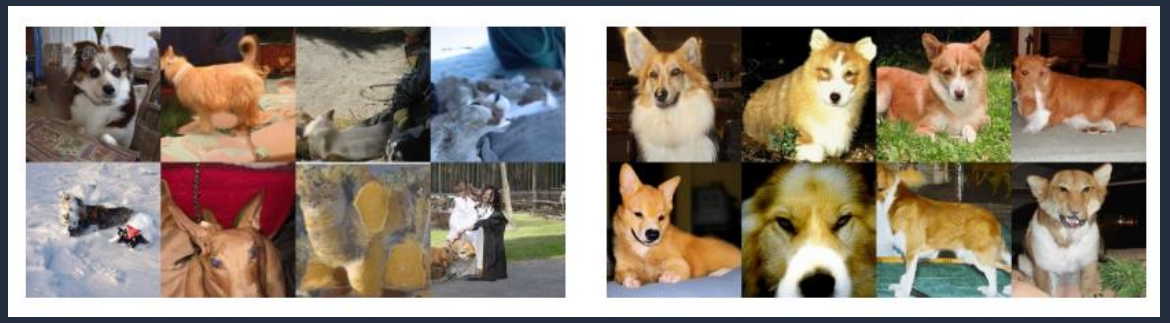


DPM-solver
(Lu et al., 2022)



Guidance

Classifier-guidance
(Dhariwal & Nichol, 2021)



Guidance with s=1

Guidance with s=10

Classifier-free guidance
(Ho & Salimans, 2022)



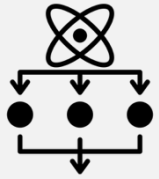
No guidance

CFG with w=3

Bringing Diffusion to Language

Why Use Diffusion Language Models?

- Autoregressive (AR) models still perform well.
- So why should we consider using diffusion for language models?



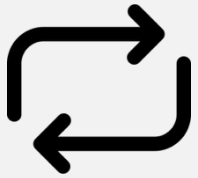
Parallel Decoding

All tokens can be generated simultaneously, utilizing more hardware



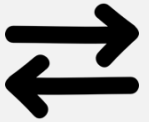
Controllability

Generation could be easily steered with Classifier guidance, Classifier-free guidance



Iterative Refinement

Allows correction of early mistakes and globally coherent outputs



Bidirectional Context

Considers full sequence context during denoising, enabling infilling

Why Use Diffusion Language Models?

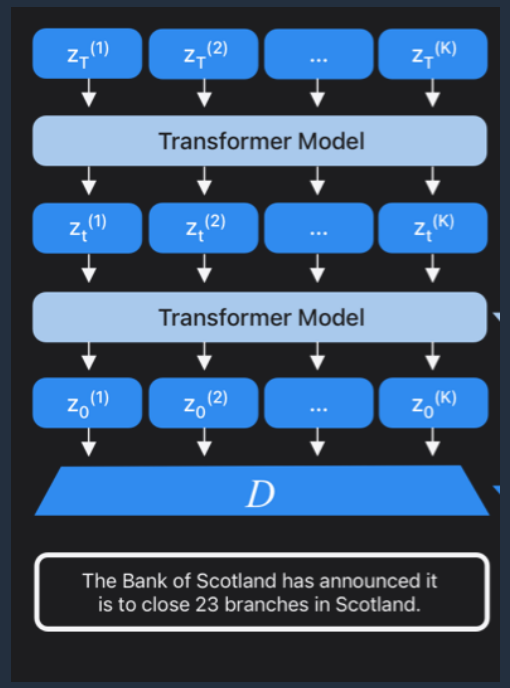
Parallel Decoding

Controllability

Iterative Refinement

Bidirectional Context

1. Parallel Decoding



https://machinelearning.apple.com/research/latent-language-diffusion-model

- Parallel generation enables simultaneous updates
- Faster generation for long sequences
- Better hardware utilization

Why Use Diffusion Language Models?

Parallel Decoding

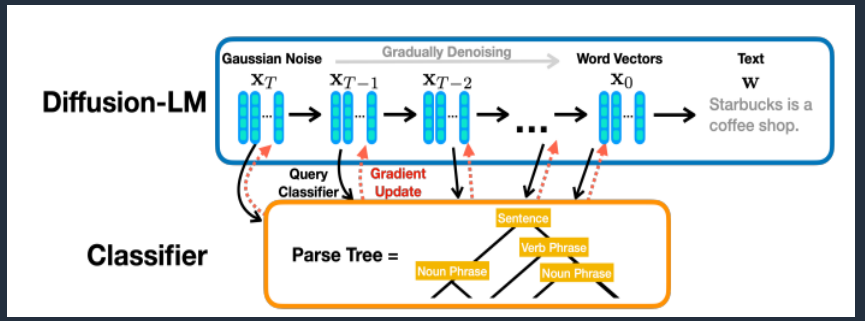
Controllability

Iterative Refinement

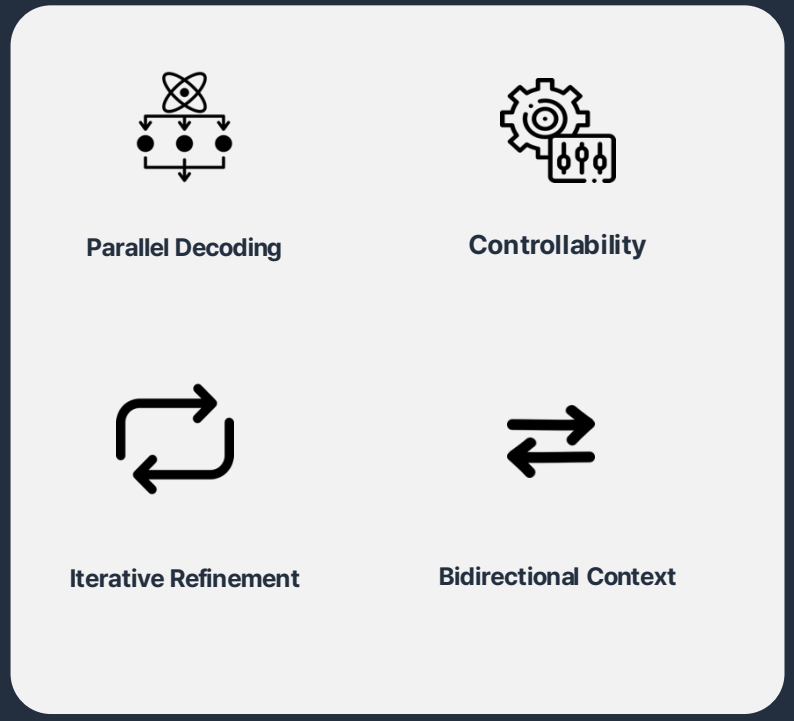
Bidirectional Context

2. Controllability

- External guidance (e.g., classifiers) can be more seamlessly integrated
- The iterative nature of diffusion progressively steers the entire sequence toward the desired attributes
- Classifier-free guidance (CFG) can also be employed to control generation without external models



Why Use Diffusion Language Models?

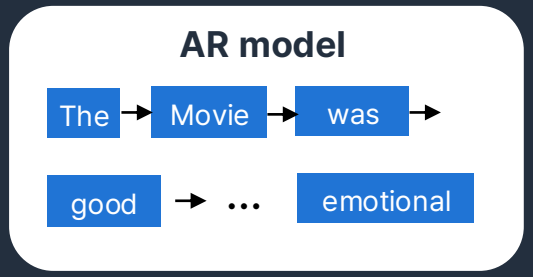


3. Iterative refinement

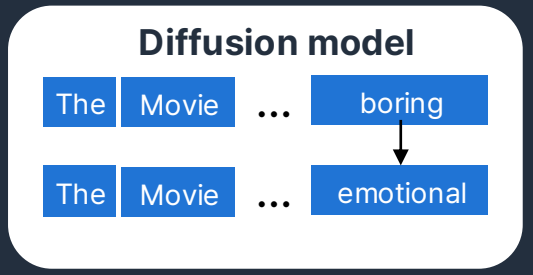
- Enable localized edits
- Support tasks like text style transfer, error correction, or controlled rewriting

Prompt: Change ‘boring’ to ‘emotional’

Target: *"The movie was good, but the ending was boring."*

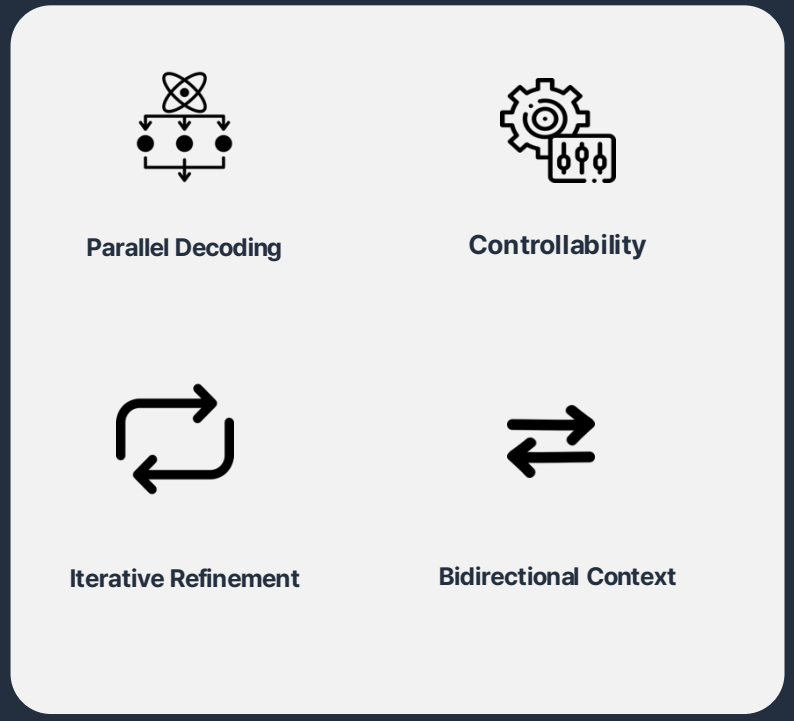


Needs to regenerate all tokens again.



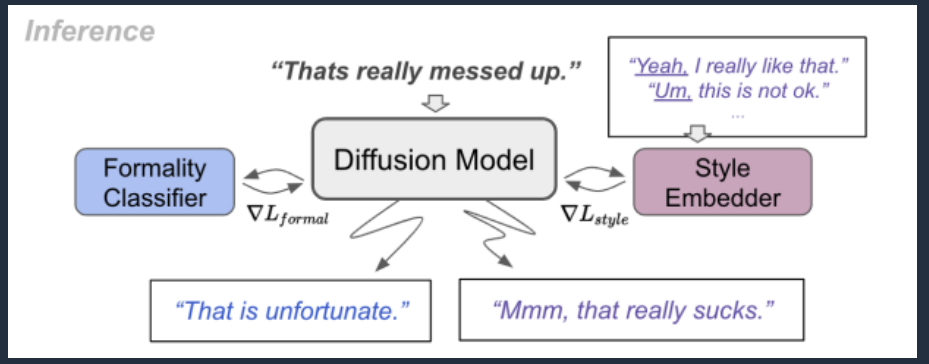
Capable of refining just the targeted token

Why Use Diffusion Language Models?



4. Bidirectional Context

- Utilize full attention which enables the model to see bidirectional context
- Helps in modeling dependencies across the entire sequence
- Useful in infilling, style transfer, rewriting

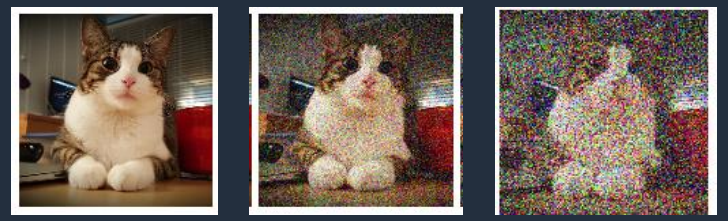


https://arxiv.org/pdf/2308.15459

Why Diffusion Doesn’t Directly Work on Text

- Unlike images, text is made of discrete tokens
  -> we can’t smoothly add noise like we do with pixels.

For image, when noised, still a valid image at each step



For text tokens

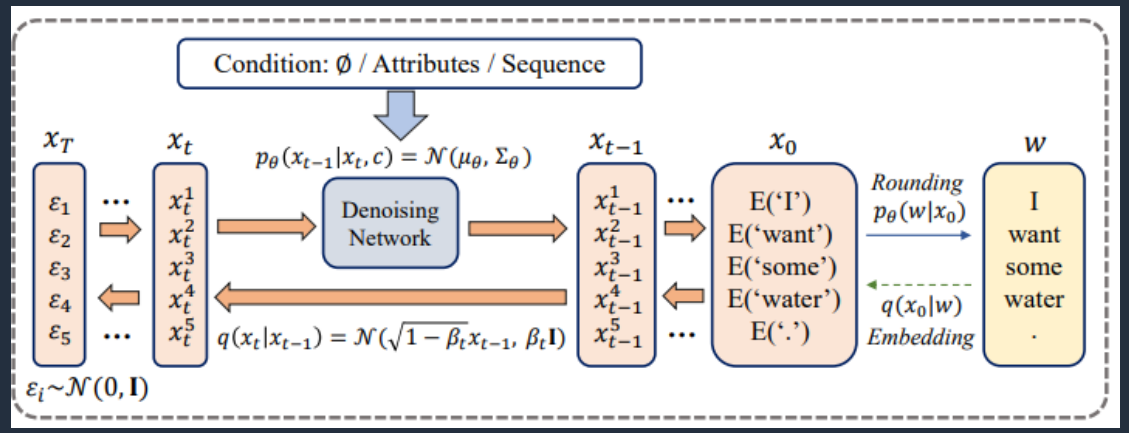
Text	Tokens
The cat sits on the mat	0, 40, 43, 13, 2, 67
↓	
? ? ? ? ? ?	0.5, 40.71, 42.1, 13.4, 1.8, 66.4
↓ Rounding?	
No image why on the love	1, 41, 42, 13, 2, 66

Dealing with Discreteness

- To apply diffusion to language, we need to bridge the gap between discrete tokens and the continuous noise-based diffusion framework.

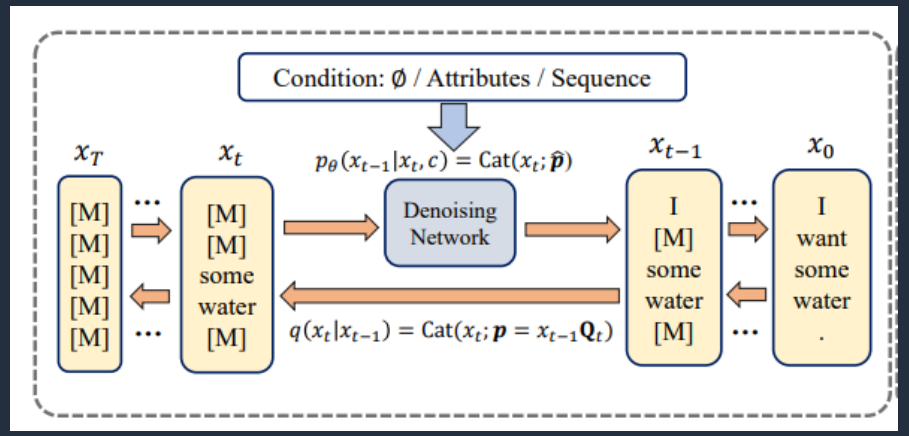
Two main paradigms

Continuous Diffusion



(https://www.ijcai.org/proceedings/2023/0750.pdf)

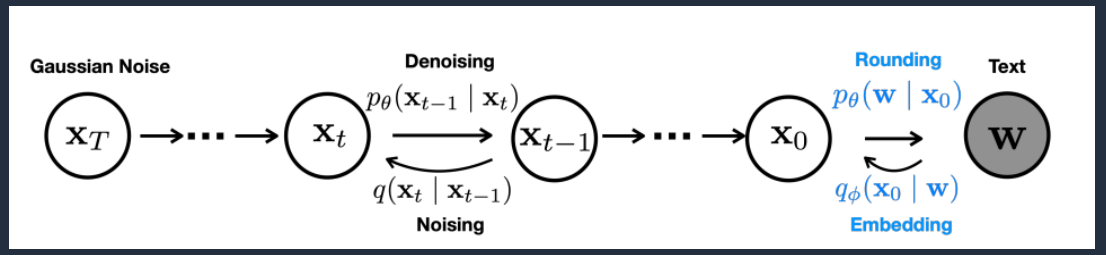
Discrete Diffusion



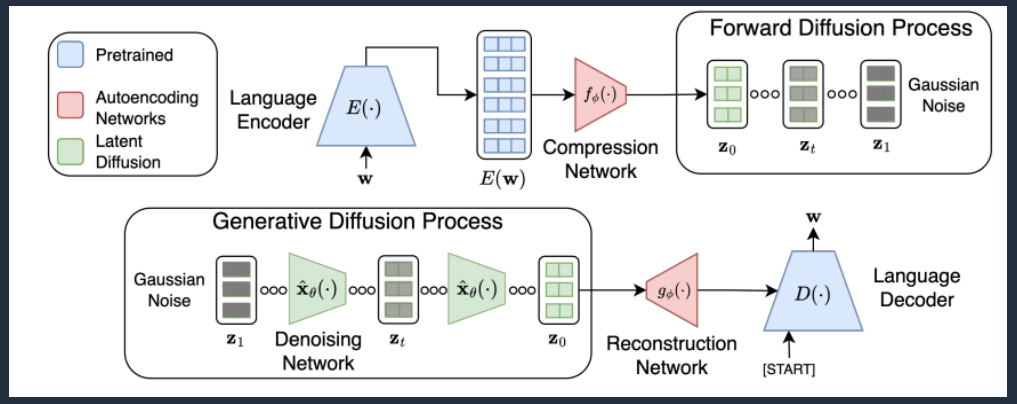
Continuous Diffusion Language Model

- Instead of adding noise directly to tokens, we embed tokens into a continuous space and apply diffusion there.

Embedding space
(Diffusion-LM, 2022)



Latent space
(LD4LG, 2022)

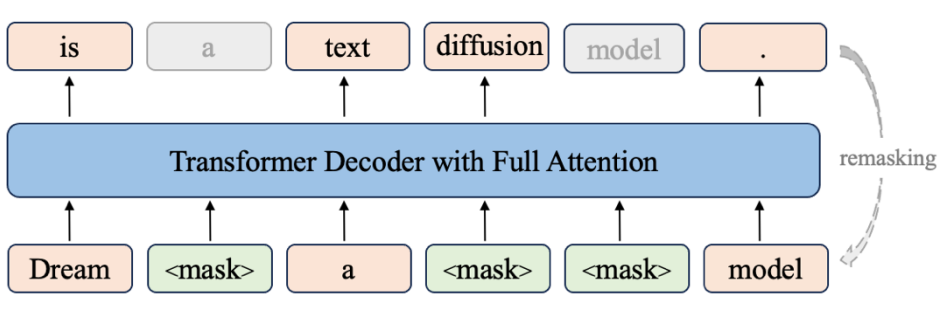


- Diffusion-LM (2022), LD4LG (2022), Genie (2023), AR-Diffusion (2024)...

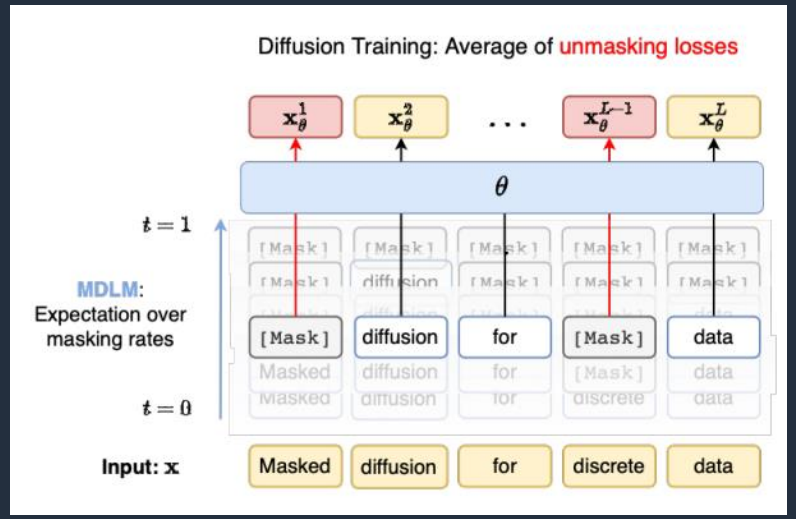
Discrete Diffusion Language Model

- Instead of embedding tokens into a continuous space, discrete diffusion models operate directly in token space by progressively corrupting the input sequence through symbolic operations.
- Random replace, mask or shuffle

Dream
(Ye et al., 2025)



MDLM
(Sahoo et al., 2024)



- DiffusionBERT (2022), MDLM (2024), SEDD (2024), LLADA (2025)...

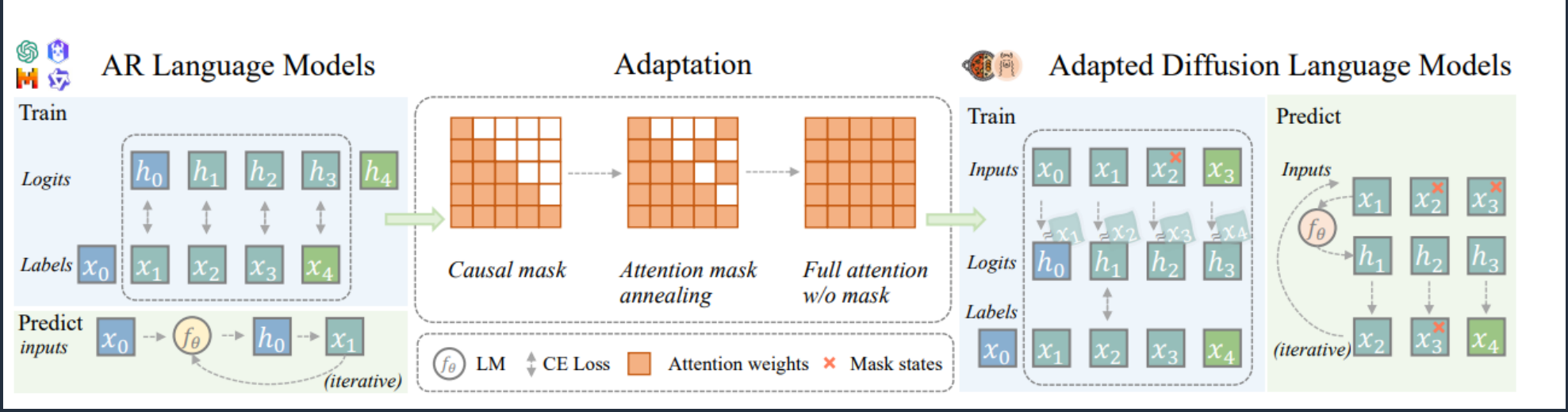
Continuous VS. Discrete Diffusion

	Continuous Diffusion LM	Discrete Diffusion LM
Space	Embedding, Latent	Token
Noising process	Continuous noising	Masking, Replacing
Pros	• Faster single-step operations (low-dim) • Classifier guidance • Image-based techniques are applicable	• Works directly on tokens • Easier alignment with existing models • Simplified training objective
Cons	• Training instability • Scaling to large corpora remains unproven • Requires encoder/decoder modules	• Classifier guidance is limited • Transition itself is not differentiable

Model architecture

- Still uses transformer-based architecture
- Attention masking is different: Causal attention -> Full attention

DiffuLLaMA (2025)



Diffusion Language Models Today

Fluency

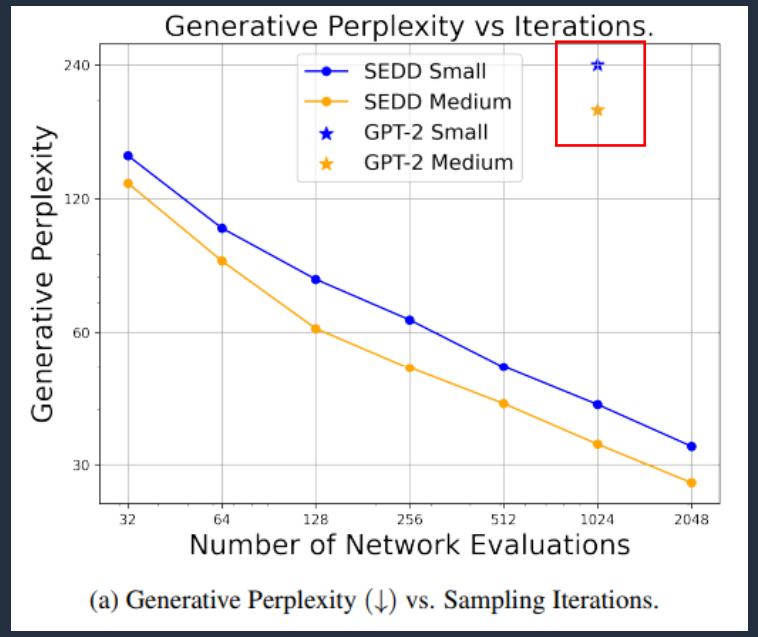
- One of the major challenges in diffusion language models has been limited fluency, as parallel generation can disrupt token-level coherence.
- However, recent advances in discrete score matching theory have significantly mitigated this issue, leading to substantial improvements in generation quality.

MDLM (2024)

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

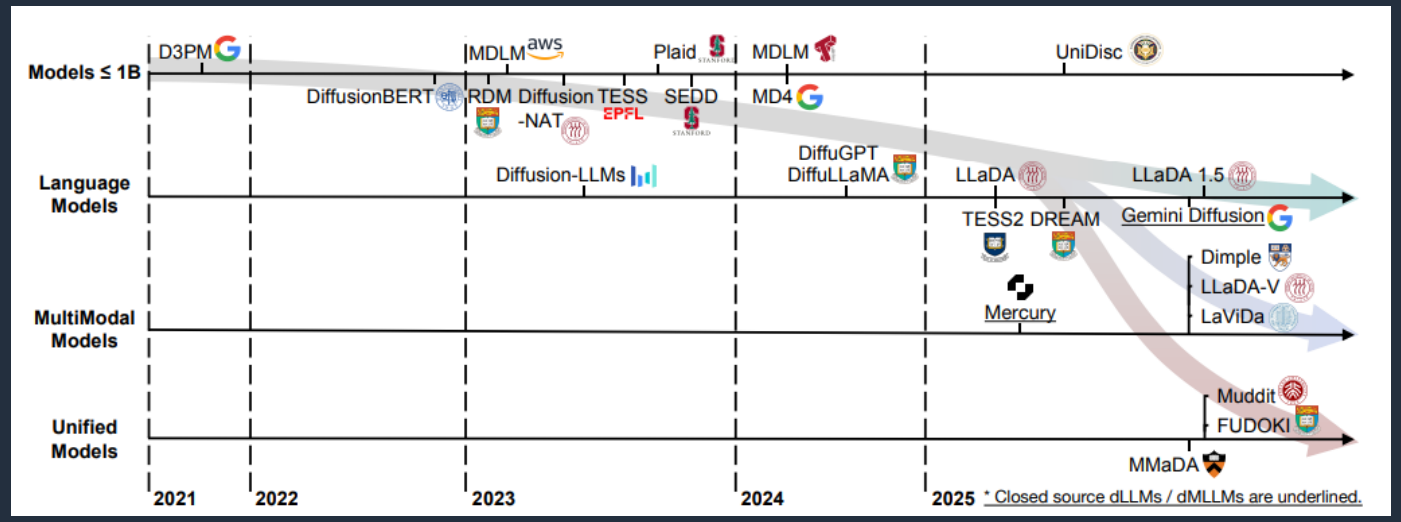
SEDD (2024)



(a) Generative Perplexity (↓) vs. Sampling Iterations.

Scalability

- Due to the high training cost and instability, diffusion language models have not been scaled as effectively as autoregressive models
- However, since the introduction of DiffuLLaMA and LLaDA, several works have emerged to tackle these limitations.



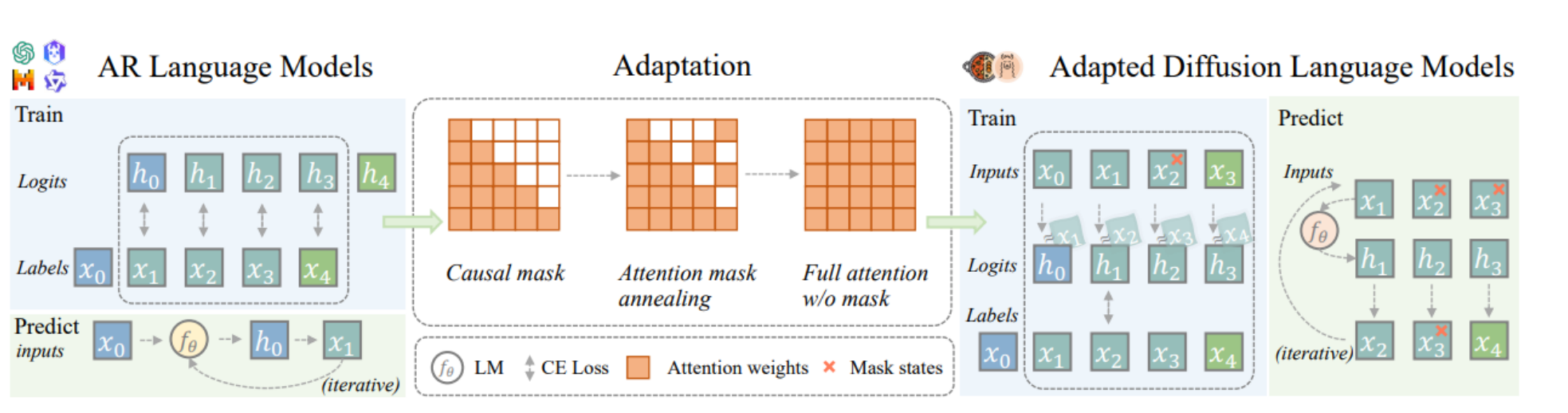
	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*
Model	Diffusion	AR	AR
Training tokens	2.3T	15T	2T
			General
MMLU	**65.9** (5)	65.4 (5)	45.9 (5)
BBH	49.8 (3)	**57.6** (3)	37.3 (3)
ARC-C	47.9 (0)	**53.1** (0)	46.3 (0)
Hellaswag	72.5 (0)	**79.1** (0)	76.0 (0)
TruthfulQA	**46.4** (0)	44.0 (0)	39.0 (0)
WinoGrande	74.8 (5)	**77.3** (5)	72.5 (5)
PIQA	74.4 (0)	**80.6** (0)	79.1 (0)

Scalability

Still, models larger than 13B have yet to be developed, and further advancements are needed.

Future directions may include:

- **Adapting** large autoregressive models for diffusion
- Incorporating **Mixture-of-Experts** (MoE) architectures
- Exploring scalable training frameworks or memory-efficient sampling methods



DiffuLLaMA (2025)

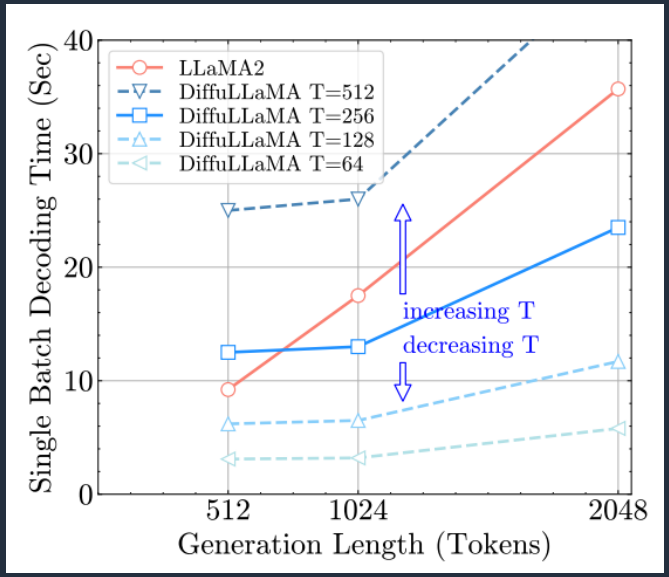
Decoding Speed

Parallel generation gives diffusion models the potential to surpass autoregressive models in speed.

However, they are not yet faster in practice, primarily due to:

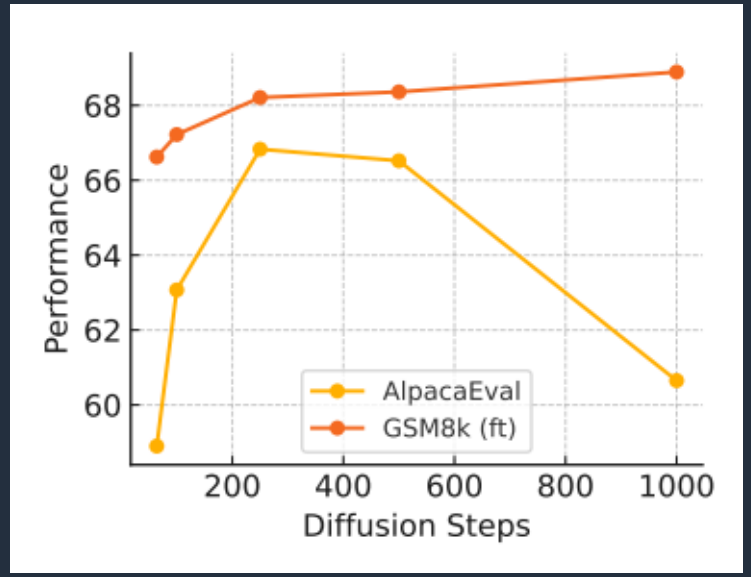
- The need for **multiple iterative steps** to produce stable outputs
- The **absence of KV-caching** mechanisms, as diffusion models typically rely on full attention at every step.

DiffuLLaMA (2025)



https://arxiv.org/pdf/2410.17891

TESS2 (2025)



https://arxiv.org/pdf/2502.13917

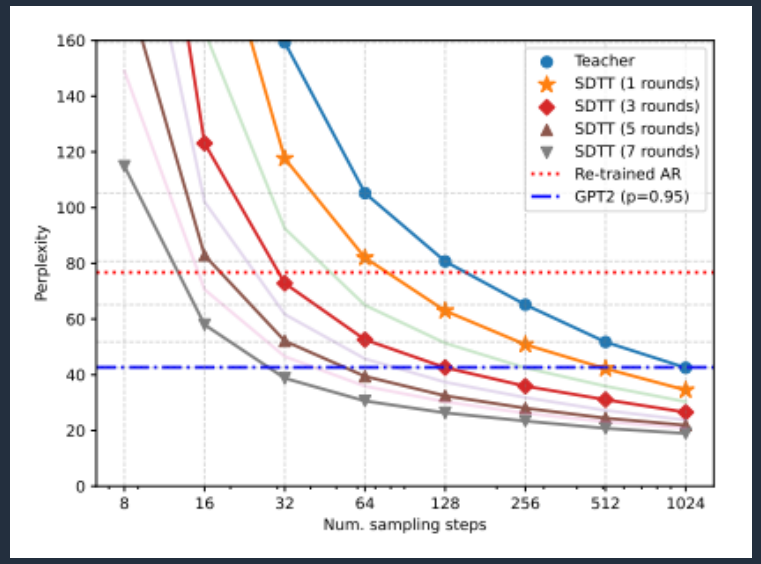
Decoding Speed

While models like Gemini Diffusion and Mercury have demonstrated impressive speed, their exact approaches remain unclear.

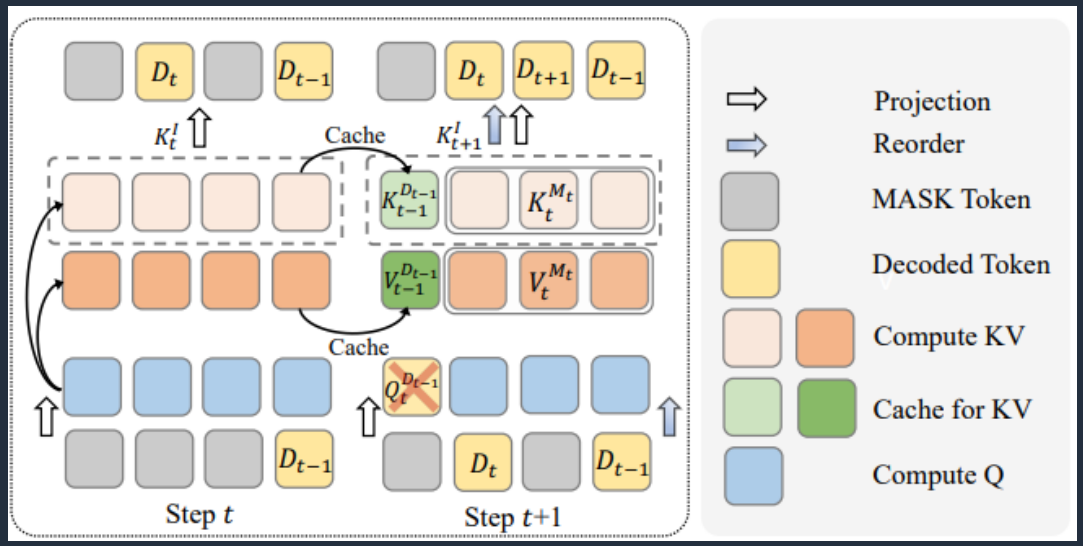
This leaves significant room for improvement in areas such as:

- Distillation techniques for faster inference
- Introducing KV-caching mechanisms for diffusion models

SDTT (2025)



dKV-cache (2025)

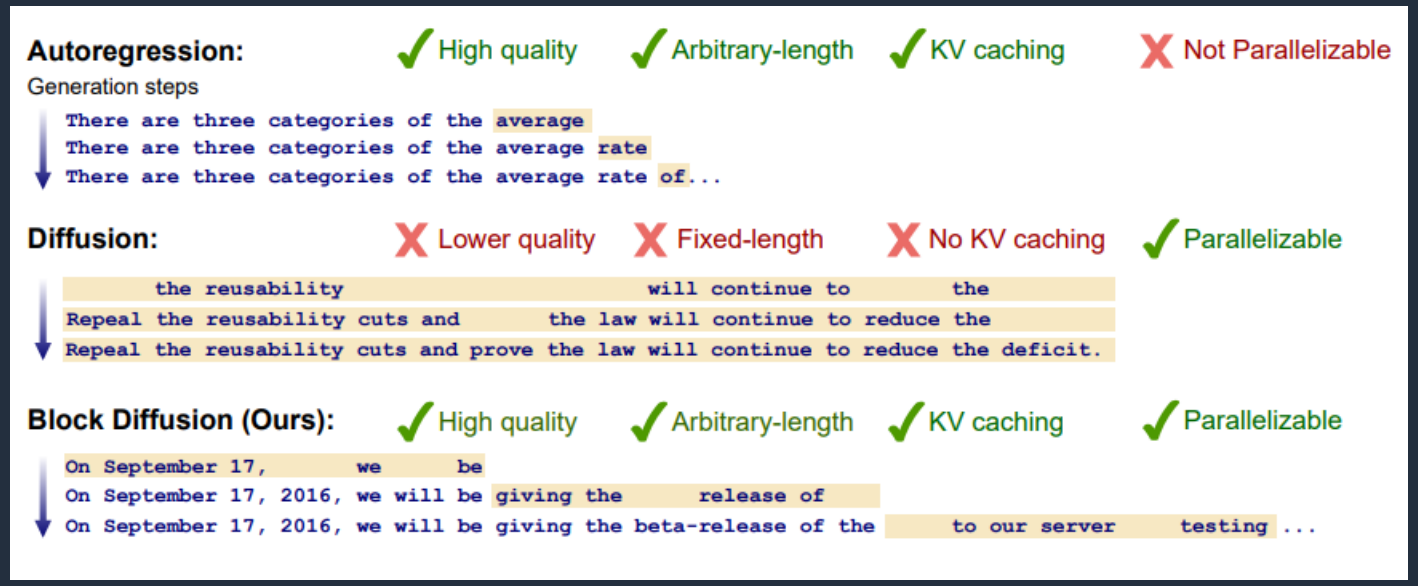


Generation Length

- Due to the constraints of parallel generation, diffusion language models are typically limited to fixed-length outputs
- This limits their efficiency, especially for short sequences or variable-length generation.

- Recent works have begun addressing this limitation through block-wise diffusion processes, which generate text one block at a time.

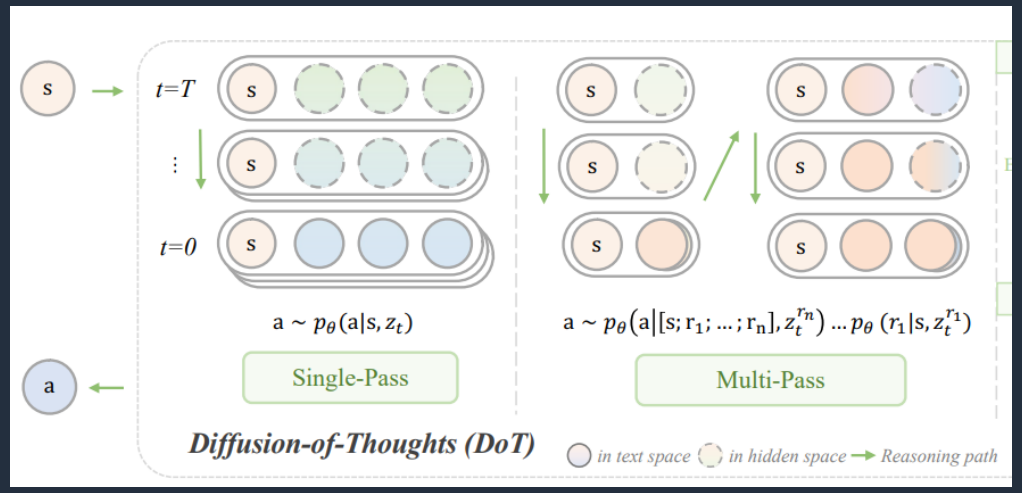
Block-diffusion (2025)



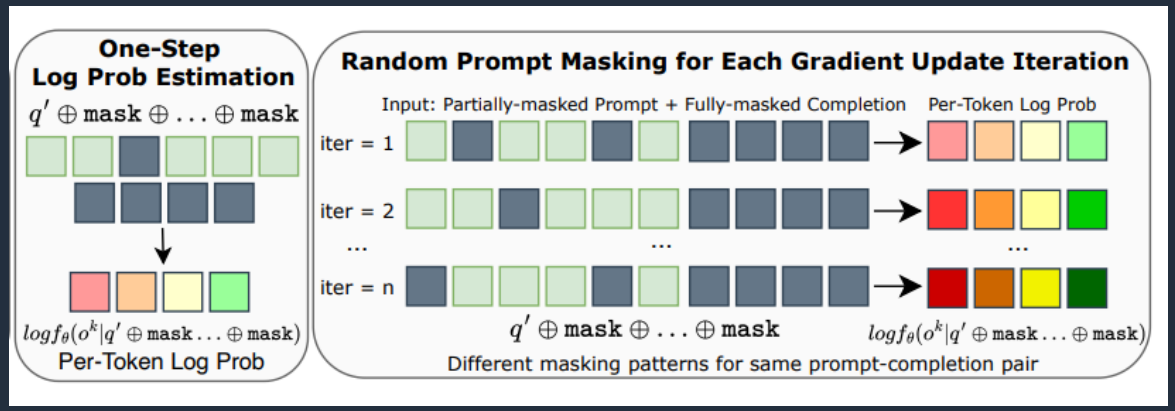
Reasoning Capability

- Diffusion models generate in parallel or by blocks, making it hard to revise their thinking in mid-generation like autoregressive mode
- Reasoning requires token-level dependencies and iterative reinforcement, which diffusion models struggle to capture.
- Recent works have begun to tackle this challenge.

DoT (2024)



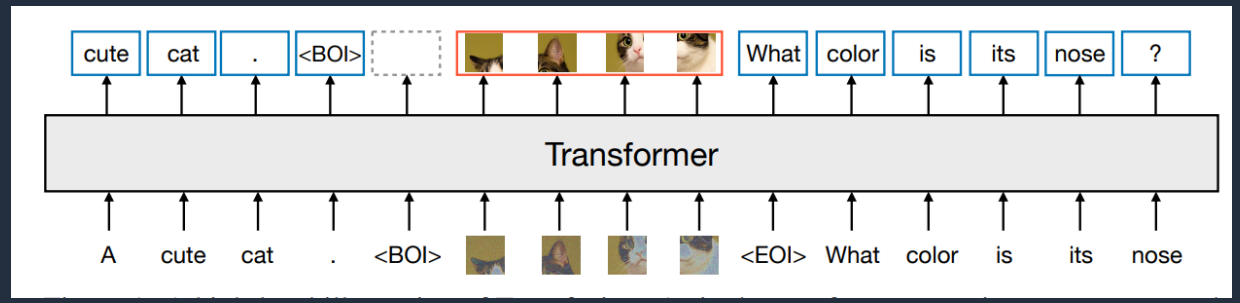
d1 (2025)



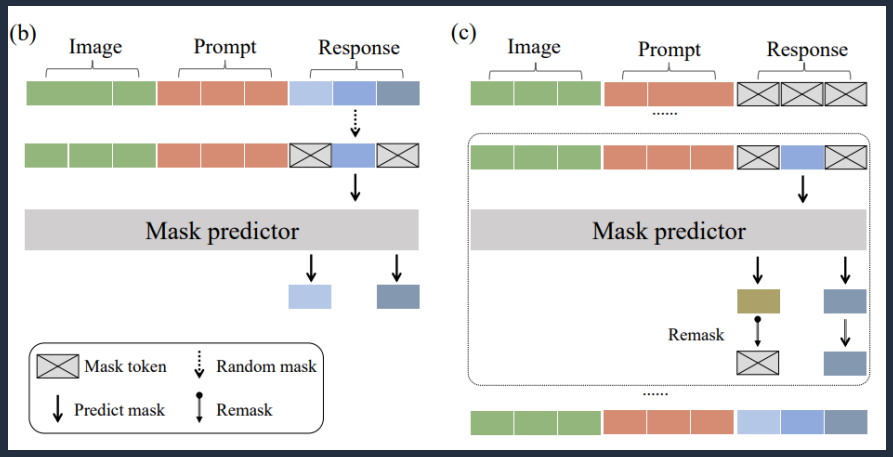
Multi-modal Input

- While current LLMs are widely used as backbones in vision-language models, the adoption of diffusion language models in this space is still in its early stages.
- Unlike autoregressive models, which generate outputs sequentially and often require modality-specific designs for audio or image generation, diffusion LMs offer parallel generation.
- This shared parallelism across modalities makes diffusion LMs a promising candidate for unified multimodal generation frameworks.

Transfusion (2024)



LLADA-V (2025)



Some other directions might include

- *Alignment:* Adapting reward modeling and RLHF techniques to the diffusion setting
- *Personalization:* Tailoring generation based on user-specific traits or preferences
- *Domain adaptation:* Fine-tuning diffusion models to perform well in specialized domains (legal, medical)
- *Decoding method:* Integrate speculative decoding, logit-based merging to improve the quality
- *Interpretability:* Investigating how diffusion updates correspond to semantic transformations
- *Pretraining/finetuning objectives: A*lternative training losses or learning strategies
- *Pruning:* Reducing inference cost by removing redundant calculation
- *Integrating RAG system:* Combining diffusion models with retrieval-augmented generation (RAG) frameworks

Seminar Schedule

14-Week Schedule

Week	Topic	Contents
Week 1 (06/25)	Introduction to Diffusion Model	
Week 2 (07/02)	Generative Modeling I	• What is generative modeling? • AE, VAE, GAN, AR
Week 3 (07/09)	Generative Modeling II	• Mathematical Formulation • Probability, ELBO, NELBO, VQ-VAE
Week 4 (07/16)	Diffusion Sampling I	• DDPM • Training, Sampling, Noise schedule
Week 5 (07/23)	Diffusion Sampling II	• SDE, Score Matching • DDIM, DPM-Solver
Week 6 (07/30)	Diffusion in Vision	• Evolution in Vision Diffusion • Latent Diffusion, DiT, ControlNet…
Week 7 (08/06)	Technical Components	• Guidance (Classifier guidance, CFG) • Self-conditioning, Distillation

14-Week Schedule

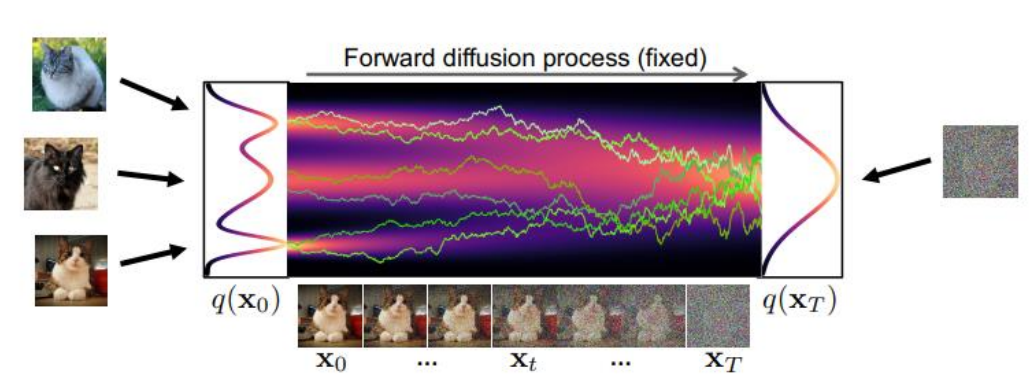
Week	Topic	Contents
Week 8 (08/13)	Diffusion Language Model I	• Continuous Diffusion Language Models • Embedding, Latent
Week 9 (08/20)	Diffusion Language Model II	• Discrete Diffusion Language Models • Discrete, Masked Diffusion Models
Week 10 (08/27)	Diffusion Language Model III	• AR VS. NAR • Block-wise Diffusion, AR like formulation
Week 11 (09/03)	Diffusion Language Model IV	• Recent Advances in DLMs • Scaling, Reasoning, Caching
Week 12 (09/10)	Deeper: Score Matching & EBM	• Score Matching • Energy-based Diffusion Models
Week 13 (09/17)	Deeper: Flow Matching	• Intro to Flow matching • Flow matching VS. Diffusion
Week 14 (09/24)	Research Trends (ICML Review)	

*See this for related papers: https://github.com/KIM-JAKE/Awesome-Diffusion-LLM

Thank you!

SDE & Score Matching

- Ideally, we'd like to model diffusion as a **continuous-time process** rather than discrete steps.
- In this view, the forward process becomes a **stochastic differential equation (SDE).**



From https://cvpr2022-tutorial-diffusion-models.github.io/

Forward Diffusion SDE:

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t$$

Reverse Generative Diffusion SDE:

$$\mathrm{d}\mathbf{x}_t = \underbrace{\left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\nabla_{\mathbf{x}_t}\log q_t(\mathbf{x}_t)\right]}_{\text{drift term}}\mathrm{d}t + \underbrace{\sqrt{\beta(t)}\,\mathrm{d}\bar{\boldsymbol{\omega}}_t}_{\text{diffusion term}}$$

"Score Function"

- We'll come back to this topic in a future session